Creating Environment

How to organize your workspace
for AI coding

madeiradev

https://madeiradev.com

CLI, Claude Code, Code Editor (IDE) 3
What is cli? 3
How to install Claude Code 4
How to install Code Editor (VS Code \w Claude Code Plugin) 5
What Is Git and Why Do You Need It? 6
Installing Git 6
Configuring Git (Mac and Windows) 7
Setting Up SSH with GitHub 8
Using Git: The Five Commands You Actually Need 10
The Daily Workflow 12
First Prompt: From Idea to Implementation 13
Developing Your App: The "Feature-First" AI Approach 15

I am so glad you took the time to go through this tutorial! I truly hope these steps — from setting up your CLI to mastering the essential Git commands — provide a helpful foundation for your AI coding journey.

It is important to acknowledge that while AI can rapidly generate complex logic, truly understanding how the code is working can be a significant challenge. When you move away from a traditional GUI and into a CLI environment where Claude Code operates, you lose the visual menus and buttons that often provide a sense of familiarity. Because AI processes your requests on remote servers and returns a finished product through a text-based interface, the underlying mechanics can feel like a "black box". This is exactly why tools like Git are essential; they allow you to "track changes" and save versions of your work, giving you a safety net to explore and learn the logic without the fear of permanently breaking your application.

If you're looking for a dedicated developer to help bring your vision to life and deliver a production-ready application, I would love to assist. You are always welcome to book a consultation with me through the link below to discuss your project:

https://calendar.app.google/WVonbTNJcLEc54hJA

My best regards,
Serge

CLI, Claude Code, Code Editor (IDE)

What is cli?

CLI — a **C**ommand **L**ine **I**nterface (Terminal, Console) is a way to control your computer by typing text commands instead of clicking buttons and icons.

Think of it this way. Your normal computer experience — clicking folders, dragging files, pressing buttons — that is a GUI (Graphical User Interface). A CLI strips all of that away. You get a blank screen with a blinking cursor, and you type commands to tell the computer what to do.

For example, instead of opening Finder and clicking through folders to find a file, you would type something like `cd Documents` to go to your Documents folder and `ls` to list what is inside.

Claude Code runs inside a CLI. You type your requests as text commands in a terminal window, and Claude Code responds by reading your project, generating code, and applying changes — all through text. No buttons to click, no menus to navigate. You type, it responds.

You do not need to be a CLI expert to use Claude Code. You need to know how to open a terminal and type a sentence. We will cover the exact steps when we get to the hands-on modules.

How to install Claude Code

What You Need Before You Start

1. An Anthropic account with a paid plan. Pro plan starts at $20/month.
2. A computer running macOS, Windows, or Linux.
3. An internet connection. All AI processing happens on Anthropic's servers. Your computer only runs the lightweight client.

Installation Steps
On Mac or Linux, open your terminal and run one command:

```
curl -fsSL https://cli.claude.com/install.sh | sh
```

On Windows, you need to install Git for Windows first then open PowerShell and run:

```
irm https://cli.claude.com/install.ps1 | iex
```

That is it. The installer downloads the Claude Code binary, places it in the right location, and configures everything automatically.

Log In
After installation, open your terminal and type:

```
claude
```

The first time you run this command, Claude Code opens your default web browser and asks you to sign in to your Anthropic account. Follow the prompts to authorize the CLI. You only need to do this once.

Verify It Worked
Type this in your terminal:

```
claude --version
```

If you see a version number, you are good to go.

Watch Out Security researchers discovered fake Claude Code installation pages. Only install from the official source at code.claude.com. Never download Claude Code from third-party websites.

How to install Code Editor (VS Code \w Claude Code Plugin)

VS Code (Visual Studio Code) is a free code editor made by Microsoft. It is the most popular code editor in the world, and it is where you will do most of your work in this course.

Go to code.visualstudio.com. The website detects your operating system automatically and shows the right download button.

- Open VS Code.
- Press `Cmd+Shift+X` (Mac) or `Ctrl+Shift+X` (Windows/Linux) to open the Extensions panel.
- Search for "Claude Code." You will see a few results — make sure you install the one published by Anthropic. Look for the verified publisher badge.
- Click **Install**.
- If the extension does not appear after installation, restart VS Code.

What Is Git and Why Do You Need It?

Git is a version control system. Think of it like "Track Changes" in Google Docs, but for code. Every time you save a version of your project, Git remembers it. If something breaks, you can go back to any previous version.

GitHub is a website that stores your Git projects online. It is like Google Drive for code. Your code lives on your computer (local) and on GitHub (remote). Git is the tool that keeps them in sync.

Why SSH? SSH (Secure Shell) is a way to prove your identity to GitHub without typing your username and password every time you push code. You create a pair of digital keys — one stays on your computer (private key), one goes on GitHub (public key). When they match, GitHub knows it is you.

Installing Git

On Mac
Open your Terminal (search for "Terminal" in Spotlight). Type:

```
git --version
```

If Git is already installed, you will see a version number. If not, macOS will prompt you to install the Xcode Command Line Tools. Click **Install** and wait — it takes a few minutes.
Alternatively, if you use Homebrew (a Mac package manager):

```
brew install git
```

On Windows
Go to git-scm.com/download/win and the download will start automatically. Git Run the installer and accept the default settings. Make sure "Git Bash" is checked during installation — this gives you a terminal that works like Mac/Linux.
After installation, open **Git Bash** (search for it in the Start menu) and type:

```
git --version
```

You should see a version number.

Configuring Git (Mac and Windows)

Before you use Git, tell it who you are. Open your terminal (Terminal on Mac, Git Bash on Windows) and type:

```
git config --global user.name "Your Full Name"
```

```
git config --global user.email "your@email.com"
```

!!! Use the same email you will use for your GitHub account.

Setting Up SSH with GitHub

Step 1 — Create a GitHub Account

Go to github.com and sign up. It is free.

Step 2 — Generate Your SSH Key

Open your terminal (Terminal on Mac, Git Bash on Windows) and type:

```
ssh-keygen -t ed25519 -C "your@email.com"
```

Replace `your@email.com` with the email you used on GitHub.
You will be prompted for a file location — press Enter to accept the default. You will then be prompted for a passphrase. You can press Enter for no passphrase (fine if you are the only one who uses your computer) or type one for extra security.

Step 3 — Start the SSH Agent

On Mac:

```
eval "$(ssh-agent -s)"
```

```
ssh-add ~/.ssh/id_ed25519
```

If you are on macOS Sierra 10.12.2 or later, you will also need to modify your `~/.ssh/config` file to automatically load keys into the ssh-agent and store passphrases in your keychain. Create or edit the file:

```
nano ~/.ssh/config
```

Add these lines:

```
Host *
  AddKeysToAgent yes
  UseKeychain yes
  IdentityFile ~/.ssh/id_ed25519
```

Save and exit (`Ctrl+X`, then `Y`, then `Enter`).

On Windows (Git Bash):

```
eval "$(ssh-agent -s)"
```

```
ssh-add ~/.ssh/id_ed25519
```

Step 4 — Copy Your Public Key

On Mac:

```
pbcopy < ~/.ssh/id_ed25519.pub
```

This copies the key to your clipboard silently.

On Windows (Git Bash):

```
cat ~/.ssh/id_ed25519.pub | clip
```

Step 5 — Add the Key to GitHub

1. Go to github.com and log in.
2. Click your profile picture (top right) → **Settings**.
3. In the left sidebar, click **SSH and GPG keys**.
4. Click **New SSH key**.
5. Give it a title (like "My MacBook" or "Work PC").
6. Paste the key you copied.
7. Click **Add SSH key**.

Step 6 — Test the Connection

```
ssh -T git@github.com
```

You may be asked if you want to continue connecting — type "yes." Mac Install Guide If it works, you will see:

```
Hi YourUsername! You've successfully authenticated, but GitHub does
not provide shell access.
```

That message means everything is set up correctly.

Using Git: The Five Commands You Actually Need

You do not need to memorize dozens of Git commands. Here are the five that cover 90% of your daily work:

`git init` — Start Tracking a Project

Navigate to your project folder and run:

```
cd path/to/your/project
```

```
git init
```

This tells Git to start watching this folder for changes.

`git add .` — Stage Your Changes

```
git add .
```

The dot means "everything." This tells Git: "Here are the files I want to include in my next save."

`git commit -m "message"` — Save a Version

```
git commit -m "Add login page"
```

This saves a snapshot of your project. The message in quotes describes what changed. Write it like a short note to your future self.

`git remote add origin` — Connect to GitHub

After creating a new repository on GitHub (click the "+" icon on GitHub and follow the prompts), link your local project to it:

```
git remote add origin git@github.com:YourUsername/your-repo-name.git
```

You only do this once per project. Notice the `git@github.com:` — this is the SSH format. That is why we set up SSH.

`git push` — Send Your Code to GitHub

```
git push -u origin main
```

This uploads your code to GitHub. After the first push, you can use `git push` without the extra flags.

The Daily Workflow

Once everything is set up, your daily routine looks like this:

Make changes to your code

↓

```
git add .
```

↓

```
git commit -m "Describe what you changed"
```

↓

```
git push
```

That is it. Three commands to save your work and back it up to GitHub.

First Prompt: From Idea to Implementation

Once your environment is set up with **Claude Code** and **VS Code**, you are ready for your first execution. Follow this structured workflow to ensure high-quality code generation.

Prompt Preparing (Templates)

Don't just ask for "a login page." Use a template to give the AI context. A good prompt includes:

- **Role:** "You are an expert React developer."
- **Task:** "Create a responsive login component."
- **Constraints:** "Use Tailwind CSS and ensure it is accessible (ARIA labels)."
- **Context:** "This will be used in a Vite-based project."

Execution

Run **Claude Code** in your terminal. Paste your prompt. Claude will read your files, generate the code, and apply changes directly to your project. Because it has access to your file system, it can see your existing folder structure and maintain consistency.

Checking

Open **VS Code** to review the files Claude modified.

- Does the code follow your project's naming conventions?
- Are the imports correct?
- Run your local development server to see the change in the browser.

Debugging

If the code throws an error, copy the error message from the console and paste it back into the Claude Code terminal. Use a prompt like:

> "The component I just created is throwing a ReferenceError. Here is the log. Please fix it." Claude will analyze the code it just wrote, identify the bug, and apply a fix.

Commit

Once the feature is working, use **Git** to save your progress.

1. **Stage:**

```
git add .
```

2. **Save:**

```
git commit -m "Add responsive login component with validation"
```

3. **Sync:**

```
git push
```

Developing Your App: The "Feature-First" AI Approach

Working with AI is most effective when you break a large "Business Requirement" into small, manageable chunks. This prevents the AI from becoming overwhelmed and reduces errors.

How to Decompose Your Business Requirement

Instead of asking the AI to "Build a project management tool," decompose it into **Micro-Features**:

Step	Goal	AI Prompt Example
1. Data Schema	Define the structure.	"Define a Prisma schema for a Task entity with titles, status, and due dates."
2. API/Logic	Create the backend.	"Create a Next.js API route to handle POST requests for new tasks."
3. UI Component	Build the interface.	"Build a 'TaskCard' component that displays the title and a status badge."
4. Integration	Connect UI to Data.	"Update the TaskCard to trigger the API route when the status is toggled."

The "Loop" Workflow

For every sub-feature identified above, follow the **Daily Workflow** mentioned in your guide:

1. **Instruction:** Give the AI the specific sub-task.
2. **Review:** Check the generated code in VS Code.
3. **Test:** Ensure the micro-feature works in isolation.
4. **Save to GitHub**

By decomposing requirements, you ensure that if something breaks, you can use Git to "go back to any previous version" without losing the entire project.